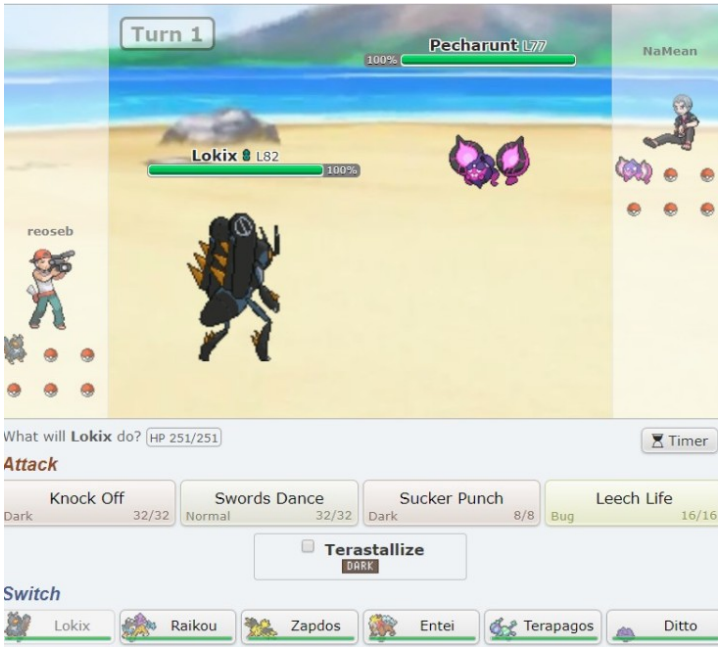
Projet de fin de module : Simulateur de combat Pokemon

Programmation Orientée Objet avec Java – Coda 1 ère année – 2026

Description du projet

L'objectif du projet est de coder l'interface graphique d'un simulateur de combat Pokemon en JavaFX
Vous devrez modéliser les différentes mécaniques de jeu en programmation orientée objet.
Le projet s'inspirera grandement de l'interface Pokemon Showdown



Déroulement

Pokémon utilise un système de combat au tour par tour ou 2 joueurs s'affrontent en utilisant leur Pokemon. Chaque joueur possède une équipe de 3 Pokemons et pendant un tour de combat les Pokemon se battent en 1 contre un.

Au début d'un combat, chaque joueur choisit un des Pokemons de son équipe et l'envoie combattre celui de l'adversaire
Chacun son tour, chaque pokemon lance une attaque sur le pokemon adverse pour lui infliger des dégâts et faire baisser ses HP. Quand les HP d'un Pokemon tombent à 0, celui ci est KO et est remplacé par un autre pokemon de l'equipe. Un joueur dont tout les pokemons sont KO à perdu.

Caractéristiques d'un Pokemon

Un pokemon possède les caractéristiques suivantes:

- Un nom
- des points de vie (HP)
- Attaque (force des attaques physiques)
- Attaque spéciale (force des attaques spéciales)
- Défense
- Défense spéciale
- Vitesse

Un Pokemon possède entre **1 et 4 attaques** qu'il peut utiliser pendant les combats

Un Pokemon ne peut utiliser qu'une liste prédéfinie d'attaques

Un Pokemon possède également entre **1 et 2 types** qui déterminent les dégâts reçus par certaines attaques

Il vous sera demandé dans votre projet de pouvoir **créer au moins 10 Pokemons différents.** Vous devrez également penser le système flexible afin de pouvoir facilement ajouter des Pokemons sans avoir à répéter trop de code.

Vous pouvez créer les Pokemons de votre choix ou même les inventer. Vous pouvez trouver une liste de Pokemons sur le lien suivant et utiliser les données officielles pour les statistiques des Pokemons

https://www.pokebip.com/pokedex/soleil-lune

Interface graphique

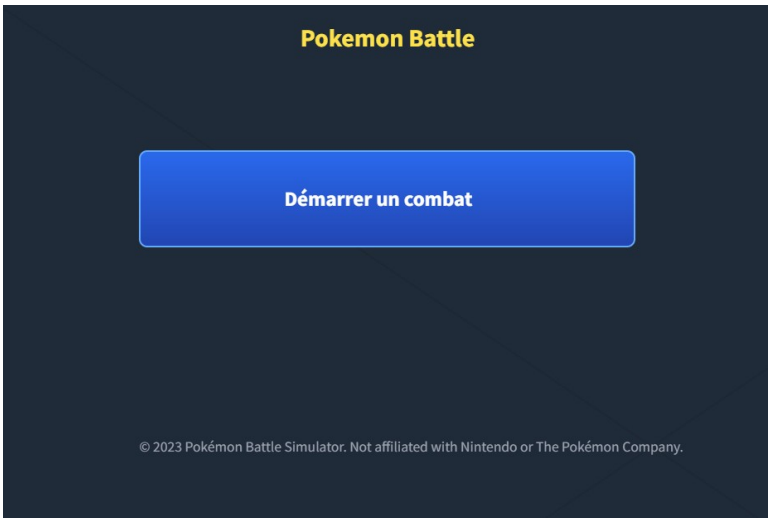
Le projet devra contenir les interfaces graphiques suivantes:

- Un menu d'accueil
- Une interface permettant de créer une équipe de 3 Pokemons
- Une interface de combat pokemon contre un ordinateur

Le design est libre. Il faudra juste que les interfaces possèdent les fonctionnalités demandées. Des points bonus pourront vous être accordés si un effort de design est réalisé

Menu d'accueil:

Le menu d'accueil est simplement un écran de présentation avec un bouton permettant de créer une équipe. Vous devez aussi afficher dans un texte les crédits du projet avec le nom et prenom des membres du projet

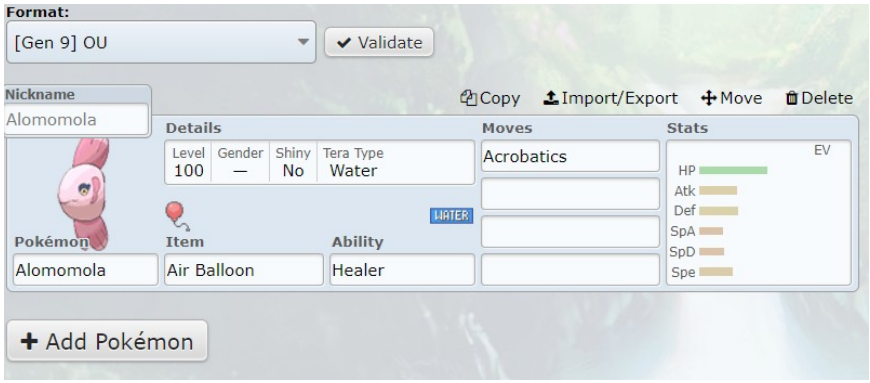


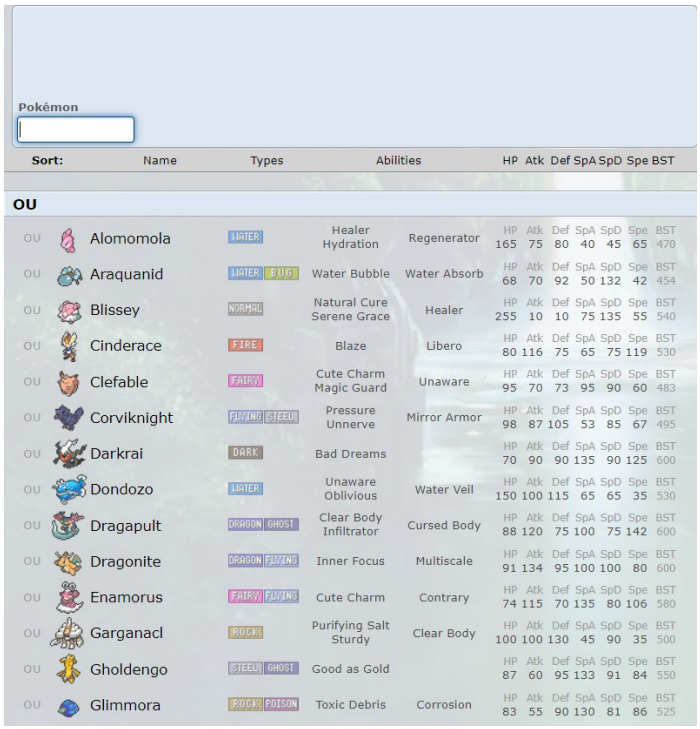
Créer son équipe de pokemon:

Avant de démarrer un combat, vous devez afficher une interface pour créer une équipe de Pokemon
Vous devez pouvoir créer une équipe de 3 Pokemons minimum (jusqu'à 6 si vous voulez)

L'interface doit permettre :

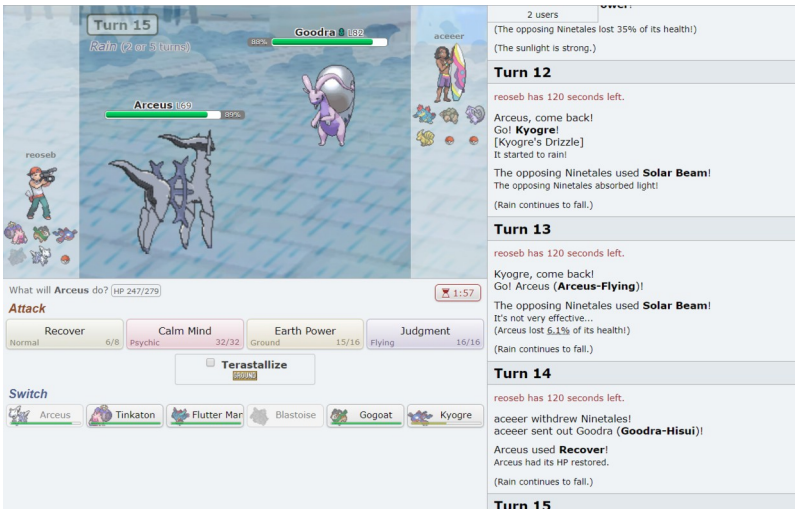
- d'ajouter un Pokemon à votre équipe. Il est possible d'avoir plusieurs fois le même pokemon
- de retirer un Pokemon de l'équipe
- de choisir les attaques d'un Pokemon
- de choisir l'objet tenu par un Pokemon et sa description
- de démarrer un combat si l'équipe est complete





Il ne vous est pas demandé de faire une interface très élaboré graphiquement. Pour l'ajout de Pokemon, une simple liste déroulante suffit. Les interfaces sont montrées à titre indicatif

Déroulement d'un combat



L'interface doit permettre d'affronter un autre joueur au tour par tour. L'écran doit afficher les éléments suivants:

- Les Pokemons actifs des 2 joueurs
- Les Pokemons de votre équipe
- Les attaques de votre Pokemon actif
- Un historique du combat
- Le statut des Pokemon
- Les HP des 2 Pokemons
- Les types des 2 Pokemons
- Le nom et éventuellement l'image du Pokemon

Dans la version de base du projet, le joueur devra affronter l'équipe de Pokemon de l'ordinateur (CPU). L'équipe du CPU peut être aléatoire.

Il doit être possible d'effectuer les actions suivantes sur l'interface

- choisir l'attaque au prochain tour
- changer de Pokemon actif. **Il n'est pas possible de lancer une attaque pendant le tour d'un changement de Pokemon**

L'historique des combats doit afficher les choses suivantes :

- Le numéro du tour
- Les attaques utilisées par les 2 Pokemons
- Les dégâts infligés
- l'efficacité des attaques : Efficace ou peu efficace
- Les effets des statuts et des talents

- Si un Pokemon est KO
- Les changements de Pokemon
- Le vainqueur du combat

L'affichage du combat doit se mettre à jour en temps réel lors des tours
L'ordinateur choisit ses attaques aléatoirement

Statistiques des Pokemons

Les statistiques influencent les combats de la manière suivante :

HP : Point de vie du Pokemon.
Attaque : Détermine la puissance des attaques physiques
Attaque Spéciale : Détermine la puissance des attaques spéciales
Défense : Réduit les dégâts reçus des attaques physiques
Défense spéciale : Réduit les dégâts reçus des attaques spéciales
Vitesse : Le pokemon qui à la plus grande vitesse attaque le premier à chaque tour. En cas d'égalité, le choix est aléatoire

La formule de dégât d'une attaque est la suivante :

<u>Attaque physique :</u>

Puissance de l'attaque physique *
(Attaque / Defense de la cible) * mutliplier de type * random (0.85 , 1)

<u>Attaque Spéciale:</u>

Puissance de l'attaque spéciale *
(Attaque spéciale / Defense spéciale de la cible) * mutliplier de type * random (0.85 , 1)

Attaque et types des Pokemons

Les attaques des Pokemons possèdent les caractéristiques suivantes:
- Une puissance
- Un nom
- Un type
- Peuvent être des attaques physiques ou spéciales.
- Peuvent avoir des effets secondaires

Les pokemons possèdent entre un et 2 types qui influence les dégâts reçus lors des combats.
Chaque type de pokemon possède des forces et des faiblesses envers d'autres types
Lorsqu'un Pokemon subit une attaque, les dégâts infligés sont multipliés par un multiplicateur qui dépend du type du de l'attaque et du Pokemon attaqué

Exemple:

L'attaque lance flamme est du type Feu et inflige 50 dégats.

Si le pokemon adverse est du type Eau, l'attaque est peu efficace et les dégats recus sont multiplié par 0.5
Si le pokemon adverse est du type plante, l'attaque est très efficace et les dégats recus sont multiplié par 2
Si le pokemon adverse à 2 faiblesses au type Feu (plante et acier par exemple), les dégats recus sont multiplié par 4
Si le pokemon adverse à 2 types résistant au type Feu (Eau et Pierre), les dégats recus sont multiplié par 0.25
Certains types sont immunisé contre d'autres types et les attaques n'infligent pas de dégat (Électrique sur Sol par exemple)

Le tableau suivant indique les forces et faiblesses du jeu officiel:

Attacker \ Defender	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Normal													½	0			½	
Fire		½	½	2		2						2	½		½		2	
Water		2	½	½					2				2		½			
Grass		½	2	½				½	2	½		½	2		½		½	
Electric			2	½	½				0	2					½			
Ice		½	½	2		½			2	2					2		½	
Fighting	2					2		½		½	½	½	2	0		2	2	½
Poison				2				½	½				½	½			0	2
Ground		2		½	2			2		0		½	2				2	
Flying				2	½		2					2	½				½	
Psychic							2	2			½					0	½	
Bug		½		2			½	½		½	2			½		2	½	½
Rock		2				2	½		½	2		2					½	
Ghost	0										2			2		½		
Dragon															2		½	0
Dark							½				2			2		½		½
Steel		½	½		½	2							2				½	2
Fairy		½					2	½							2	2	½	

Vous devez implémenter **au moins 8 types sur la liste**. Néanmoins vous devez penser une architecture flexible qui permet d'ajouter facilement de nouveaux type à votre système.

Vous pouvez vous inspirer de la liste d'attaque officielle :
https://www.smogon.com/dex/ss/moves/

Effets des attaques

Certaines attaques en plus d'infliger des dégâts provoquent divers effets:

Exemple :

Damoclès

Le lanceur subit 1/3 des dégâts infligés à l'adversaire.

Lance-flamme:

10 % de chance de brûler le Pokémon défenseur.

Giga-Sangsue:

Le lanceur est soigné de 1/2 des dégâts infligés à l'adversaire.

Pour le projet, il vous est demandé d'implémenter **au moins 4 attaques avec des effets secondaires différents.** Pensez votre architecture flexible qui permet d'ajouter ou de réutiliser facilement les effets des attaques

Vous pouvez vous inspirer des effets de la liste des attaques officielles
https://www.smogon.com/dex/ss/moves/

Statuts

Lors d'un combat, un Pokémon peut se voir infliger différents statuts qui altèrent ses capacités
Vous devez implémenter les 3 statuts suivant et des capacités qui permettent de les infliger

Paralysie:

Un Pokemon paralysé à 25 % chance de ne pas pouvoir attaquer pendant son tour. De plus, sa vitesse est divisée par 2.

Poison:

Un Pokemon empoisonné perds 1/8 de ses HP à la fin de chaque tour

Brulure:

Un Pokemon brulé perds 1/16 de ses HP à la fin de chaque tour. La puissance de ses attaques physiques est aussi divisée par 2.

Objets tenus par les Pokemons

Un Pokémon peut tenir un objet qui lui confère des effets passifs ou qui peut être utilisé automatiquement dans certaines conditions. Ces objets peuvent soigner, augmenter des statistiques, bloquer des altérations de statut, ou encore être consommés pour booster une attaque.

Exemple :

Restes : Restore 8 % des HP d'un Pokemon à la fin de chaque tour
Ballon : Immunité aux attaques de type Sol. L'objet est détruit si le Pokemon subit une attaque
Lunette de soleil : Augmente la puissance des attaques de type « Dark » x1.2

Implémentez au moins 5 objets avec des effets différents. Il doit être possible d'assigner les objets aux Pokemon lors de la création de l'equipe

Vous pouvez vous inspirer des objets du jeu officiel
https://www.smogon.com/dex/ss/items/

Fonctionnalités additionnelles

Il vous est demandé d'implémenter au moins 2 fonctionnalités de la liste suivante. Si vous implémentez une 3 eme fonctionnalité , des points supplémentaires vous seront accordés

- Charger les Pokemons depuis un fichier csv ou une base de données
- Talent des Pokemons
- Implémenter un système d'objet : Au lieu d'utiliser une attaque, un joueur peut utiliser un objet
- EV/IV
- Mega Evolution
- Ajouter des sons et des effets lors des combats
- IA avec machine à état
- Augmentation de statistiques
- Faire des suites de combat avec un mode Arène

Charger les Pokemons depuis un fichier ou une base de données

Les Pokemons ainsi que leur caractéristiques doivent être chargé depuis un fichier au format de votre choix ou une base de données.

Au démarrage de l'application, vous devez initialiser tout les Pokemons de votre jeu depuis ces données externes

Talents des Pokemons

Les talents sont des capacités passives propres à chaque Pokémon. Contrairement aux attaques, les talents s'activent automatiquement sous certaines conditions et peuvent influencer les combats en modifiant les statistiques, en ajoutant des effets secondaires, ou en rendant un Pokémon immunisé à certaines attaques.

Chaque Pokemon possède un talent et un talent peut être partagé par plusieurs Pokemon

Implémentez au moins 3 talents avec des fonctionnements différents (Pas juste des valeurs différentes). Pensez l'architecture flexible

Exemple

Lévitation : Immunité contre les attaques de type Sol
Brasier : Augmente la puissance des attaques feu de x1.5 si un Pokemon a perdu 2/3 de ses HP
Vaccin : Le Pokemon ne peut pas être empoisonné

Vous pouvez vous inspirer des talents officiels ou inventer les votre
https://www.pokepedia.fr/Liste_des_talents

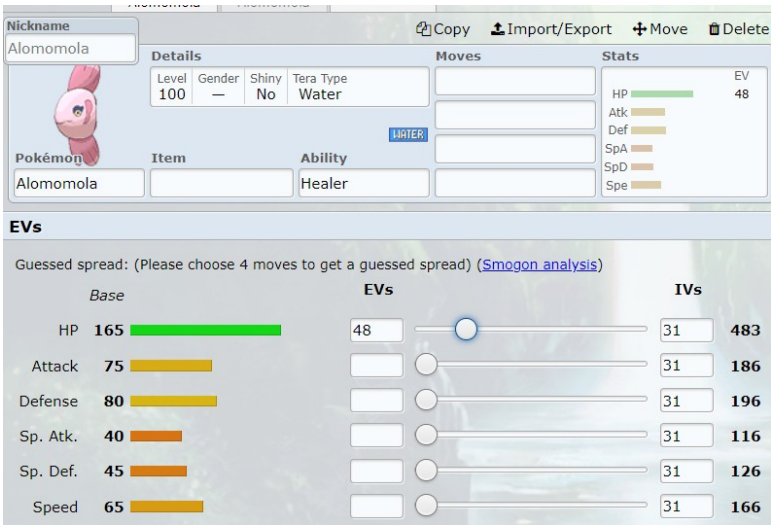
EV

Lors de l'ajout d'un Pokemon à votre équipe, ajouter une interface pour attribuer des « EV » à votre Pokemon.

Les EV sont des points supplémentaires que vous pouvez attribuer aux statistiques. Pour le projet, un joueur peut attribuer 50 EV à un Pokemon avec une limite de 20 EV par statistiques :

20 EV à l'attaque – 20 EV à la défense – 10 EV à la vitesse
50 EV à l'attaque = impossible

Vous devez utiliser des sliders JavaFX pour attribuer les EV par statistiques. Faites en sorte que les sliders gèrent les bonnes limites

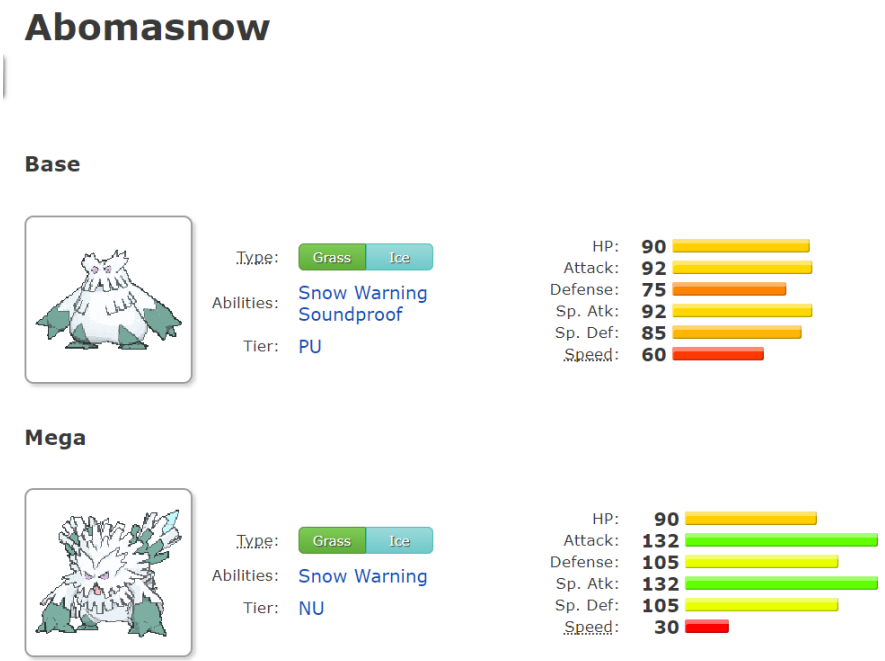


Mega Evolution

La Méga-Évolution est une transformation qui permet à certains Pokémon de devenir plus puissants en combat. Cette transformation augmente considérablement ses statistiques, modifie parfois son type ou son talent

Il n'est possible d'utiliser qu'une seule Méga Evolution par combat et par équipe. Un Pokemon qui a une Méga Evolution ne peut pas porter d'objet

Au début d'un tour, il est possible de faire Méga Evoluer le Pokemon actif. Cette action n'utilise pas de tour et le Pokemon peut attaquer directement



Vous devez implémenter au moins 3 Pokemons avec des Méga-Evolution. Ajouter un bouton dans l'interface de combat pour effectuer une Mega Evolution si cela est possible.
<u>Pensez votre architecture flexible qui permet d'ajouter facilement des Méga Evolution</u>

Ajouter des sons et des effets

Ajouter à votre projet, la possibilité de jouer des sons lors des attaques ou de l'interaction avec l'UI et afficher des animations pour certaines attaques ou certains effet. Vous pouvez vous contenter d'utiliser des GIF pour les animations

Ajouter au moins 5 occurrences de sons dans votre projet et au moins 3 effets.

Augmentation de statistique:

Certaines capacités ne sont pas des attaques mais permettent d'augmenter les statistiques du Pokemon qui la lance ou de baisser les statistiques du Pokemon adverse selon un système de niveau :

Exemple :

Agilité double la vitesse de 2 niveaux
Gonflette augmente la défense physique et l'attaque physique d'un niveau
Rugissement baisse l'attaque physique du Pokemon adverse d'un niveau
Close combat inflige des dégats et réduit d'un niveau la defense et la defense spéciale du lanceur

Le tableau suivant indique l'influence de l'augmentation de niveau sur un Pokemon

	Diminution						Base	Augmentation					
Niveau	-6	-5	-4	-3	-2	-1	0	+1	+2	+3	+4	+5	+6
Fraction	$^1/_4$	$^2/_7$	$^1/_3$	$^2/_5$	$^1/_2$	$^2/_3$	1	$^3/_2$	2	$^5/_2$	3	$^7/_2$	4
Pourcentage	25%	29%	33%	40%	50%	67%	100%	150%	200%	250%	300%	350%	400%

Implémenter ce systeme de niveau. **Les modifications de statistiques doivent être affichées sur l'affichage des combats et dans son historique. Vous devez implémenter au moins 4 attaques qui influencent les statistiques du lanceur ou de la cible**

Systeme d'objet

Implémentez un système d'objet qui peuvent être utilisé pendant les combats . Au lieu de lancer une attaque, un joueur peut utiliser un objet sur un Pokemon

Exemple d'objet :

Potion : Restaure 50 HP
Antidote : Soigne le statut Poison
Rappel : Ressuscite un Pokemon KO a 50 % de sa vie

Implémentez 4 types d'objet différents. Ajouter dans l'interface de combat, la possibilité d'ouvrir un inventaire et de sélectionner un objet.

Les objets utilisable en combat peuvent être définit par défaut ou configurable lors de la création de l'equipe

IA pour le comportement du joueur CPU

Plutôt que de choisir une attaque au hasard, développez une IA simple pour implémenter une logique de décision

Votre algorithme doit pouvoir utiliser différentes stratégie en fonction de la cible et utiliser différentes mécaniques de combat pour prendre la décision.

Votre IA doit pouvoir, par exemple, changer de Pokemon si cela est opportun, utiliser les status, s'adapter aux objets tenus ou prendre en compte les capacités des autres Pokemon de l'equipe.

Vous pouvez utiliser une machine à état par exemple mais toute autre technique même simple est valable.

Votre programme doit implémenter au moins 2 types de stratégie et analyser les differents paramètres du jeu et exploiter le plus de mécanique possible.

Une IA qui choisit seulement l'attaque la plus puissante ne suffira pas à avoir tout les points

Challenge et mode arène

Ajoutez un nouveau mode de jeu :

Au lieu de combattre un seul CPU, le joueur devra combattre un série de dresseur **qui ont des Pokemons prédéfinis** sous la forme de plusieurs combats qui se suivent .

Sur l'écran d'accueil, ajouter un bouton supplémentaire pour jouer en mode « Arène ». Le joueur passera sur un nouvel écran dans lequel il devra choisir quelle « Arène » il veut combattre parmis une liste prédéfinie, une arène étant une série de combat. Le joueur devra ensuite créer son équipe comme dans le mode normal.

En mode Arène, **si le joueur gagne un combat il passe automatiquement au combat suivant.** Entre chaque combat, vous pouvez choisir de soigner les Pokemons du joueur ou de les laisser dans le meme état que le combat précédent. Le nombre de combat d'une Arène est arbitraire et définit par votr programme et peut aller de 1 à N

Si le joueur gagne la série de combat, vous gagnez la partie. Si le joueur perds tout ses Pokemon pendant un des combats, le mode Arène s'arrête

Implémentez le système d'Arene et proposez au moins 2 types d'arène. Il doit être possible de configurer le nombre de Combat par Arène et les Pokemons de chaque combat.

Vous devez essayer de créer une architecture flexible et modulable qui permet de configurer facilement les arène de votre jeu

Critère d'évaluation

Vous serez tout d'abord évalué sur votre capacité à utiliser correctement les concepts de POO dans un projet composé de nombreuses classes.
Prenez le temps de faire les bons choix de modélisation. Respectez les bonnes pratiques de programmation orientée objet :

DRY (Don't Repeat Yourself)
Single responsability principle

Vous devez essayer de créer une architecture flexible et évolutive.
Des points vous seront retiré si vous faites de très mauvais choix de conception ou si vous ne respectez pas les bonnes pratiques de code

Le jeu est composé de nombreuses règles indépendantes et il est possible de réaliser beaucoup de taches en parallèle. Un bon développeur sait travailler en équipe, organisez vous bien avec votre binôme et découpez astucieusement le travail.

Vous serez également évalue sur votre capacité à réaliser une interface dynamique. L'architecture de vos classes doit correspondre aux besoins de l'UI. Vous pouvez développer l'UI en code Java pur ou en utilisant des fichiers FXML.

Enfin, le projet est composé de nombreuses fonctionnalités, vous devez essayer d'aller au bout des choses et de rendre un projet abouti.

Il n'y a pas une solution unique à ce projet et des architectures très différentes peuvent être tout à fait valide. Vous obtiendrez une bonne note si vous faites preuve de jugement

Évaluation du projet

Votre note dépendra des points suivants

- Le nombre de fonctionnalités implémentées
- Les interactions et le contenu de votre interface graphique
- La qualité de votre architecture

50 % de la note sera au prorata du nombre de fonctionnalités implémentées. Si vous implémentez toutes les fonctionnalités de base du projet + 2 fonctionnalité additionnelle, vous obtiendrez la moitié des points

Si vous implémentez toutes les fonctionnalités de l'interface graphique vous obtiendrez 1/4 des points

Votre architecture sera étudiée et 1/4 des points dépendent de vos choix de conception
Votre architecture doit avoir du sens et utiliser les concepts de POO au bon moment: Héritage, Encapsulation, Polymorphisme etc... Vous devez essayer d'avoir un programme extensible et évolutif.

Une utilisation trop partielle ou des utilisations manqués des concepts en POO vous feront perdre des points.
Une conception trop rigide, mal pensée ou un programme peu évolutif influencera négativement votre note

Des points bonus vous seront accordés si vous implémentez une fonctionnalité additionnelle en plus mais aussi en fonction de la qualité de votre interface (design travaillé, responsive)

Vous serez encore et toujours évalué sur la qualité de votre code et devrez mettre en place les bonnes pratiques vues lors de votre formation. A savoir :

- Des noms de variables, de fonctions et de classes claires
- Utiliser la convention camelCase
- Des fonctions de moins de 20 lignes
- Une indentation correcte
- Un code correctement organisé
- Mise en forme du code uniforme dans le projet (accolades, saut de ligne, conventions de codage)
- Des commentaires si besoin

Des petits écarts peuvent être toléré, cependant si aucun effort n'est réalisé ou si certaines pratiques ne sont pas mises en place, **vous pourrez perdre jusqu'à 5 points sur votre note finale** même si votre projet fonctionne parfaitement.

Rendu

Vous devez rendre votre projet sous la forme d'un fichier compressé .zip contenant l'ensemble de vos fichiers ou d'un lien vers un repository par email à l'adresse suivante

dany.siriphol@gmail.com

Vous avez jusqu'au **Mercredi 18 Mars 23h59** pour rendre votre projet. 2 points seront retirés à votre note par jour de retard